强化学习 L1–L10 核心归纳

从 MDP、动态规划到价值函数、策略梯度与 Actor–Critic

根据课程讲义整理

面向初学者的快速复习版

2026 年 8 月 23 日

这份笔记解决什么问题？

学到后面的算法时，前面的符号和逻辑很容易混在一起。本文只保留 L1–L10 最重要的主线，并在每个核心公式出现前先说明它在回答什么问题、为什么会有这个形式。它不是逐页复刻课件，而是一份可以反复扫读的“知识地图”。

十讲只讲了一条主线：

描述环境（MDP） ⟶ 价值关系（Bellman 方程） ⟶ 有模型：VI/PI

无模型：MC 样本估值 ⟶ 随机逼近 ⟶ TD：一步/多步目标

函数逼近：值函数/策略函数 ⟶ Actor–Critic.

本文默认的讨论范围：有限状态、有限动作、平稳 Markov 策略，以及折扣因子 $\gamma \in [0, 1)$ 的 MDP。这个范围足以覆盖课件 L1–L8 的主线；L9–L10 还会在讲义明确标出的地方补充平均奖励、未折扣与离策略情形。其中 L1–L4 主要讨论模型已知时的 Bellman 方程与动态规划；L5 开始讨论模型未知时如何用 episode 样本估计价值；L6 解释这种“边采样、边更新”的迭代为什么可能收敛；L7 则把随机逼近用于 Bellman 方程，得到 TD、Sarsa、n-step Sarsa 和 Q-learning；L8 用参数化函数替代表格并引出 DQN；L9 直接参数化策略并通过策略梯度优化；L10 把策略更新的 actor 与 TD 价值估计的 critic 组合起来。

目录

统一记号与阅读方法 **5**

1 L1：基本概念——把交互问题写成 MDP **6**

1.1 智能体与环境如何交互 6
1.2 五个基础概念 6
1.3 轨迹、回合与回报 7
1.4 Markov 性与 MDP 8

2 L2：状态价值与 Bellman 期望方程 **9**

2.1 从一次回报到平均表现 9
2.2 Bellman 方程为什么会出现 9
2.3 动作价值的 Bellman 形式 10
2.4 一个两状态例子 10
2.5 怎样求出所有状态价值 11

3 L3：最优策略与 Bellman 最优方程 **12**

3.1 从评价转向控制 12
3.2 Bellman 最优方程怎样得到 12
3.3 期望方程与最优方程不要混 13
3.4 为什么最优方程有唯一解 13
3.5 从最优价值取出最优策略 13
3.6 什么会改变最优策略 14

4 L4：价值迭代、策略迭代与截断策略迭代 **15**

4.1 先明确：这是有模型的动态规划 15
4.2 价值迭代（Value Iteration, VI） 15
4.3 策略迭代（Policy Iteration, PI） 16
4.4 截断策略迭代（Modified/Truncated PI） 16
4.5 三种算法的对比 17
4.6 实现时最值得记住的四点 17

5 L5：Monte Carlo 方法——没有模型时，用完整经历估计价值 **18**

5.1 从“知道模型”到“只拿到样本” 18
5.2 MC Basic：把策略迭代改成无模型版本 19
5.3 更有效地使用一条 episode 19
5.4 Exploring Starts：理论上好用，实践中难满足 20

5.5 ε-greedy：用软策略替代 exploring starts 20
5.6 L5 与前四讲的关系 21

6 L6：随机逼近——为什么样本更新可以逐渐靠谱 22
6.1 从普通平均到增量平均 22
6.2 Robbins–Monro：只看到带噪声的函数值，也能找根 22
6.3 步长条件：既要走得远，也要越来越稳 23
6.4 均值估计其实是 RM 的特例 23
6.5 SGD：用一个样本梯度近似真实梯度 24
6.6 BGD、MBGD 与 SGD 的区别 24
6.7 L6 对后续 RL 的意义 25

7 L7：时序差分学习——不等 episode 结束就更新 26
7.1 先看全章地图：L7 是 L2、L5、L6 的交汇点 26
7.2 基础 TD：用一步样本估计状态价值 26
7.3 TD 与 Monte Carlo：都无模型，但更新时机不同 28
7.4 基础 TD 的收敛条件 28
7.5 Sarsa：把状态价值 TD 改成动作价值 TD 29
7.6 Expected Sarsa：对下一动作先取策略期望 30
7.7 n-step Sarsa：在一步 TD 与 MC 之间调节 30
7.8 Q-learning：直接逼近最优动作价值 31
7.9 On-policy、off-policy、online、offline 不要混 32
7.10 统一视角：算法的骨架相同，只是 TD 目标不同 33
7.11 拿到一条经验时，怎样判断该用哪个目标 34

8 L8：价值函数方法——从表格走向函数逼近 35
8.1 为什么要把表格换成函数 35
8.2 把价值估计写成优化问题 35
8.3 带函数逼近的 TD 状态价值学习 36
8.4 线性 TD 最终在求什么 36
8.5 LSTD：直接估计固定点方程 37
8.6 动作价值：Sarsa 与 Q-learning 的函数版本 37
8.7 Deep Q-learning：两个网络加经验回放 38

9 L9：策略梯度方法——直接优化参数化策略 39
9.1 从策略表格到策略函数 39
9.2 两个标量性能指标 39

9.3 策略梯度定理 40
9.4 统一公式中的状态分布不能省略 40
9.5 REINFORCE：用完整回报估计动作价值 41

10 L10：Actor–Critic——让策略更新与价值估计协同工作 42
10.1 Actor 与 critic 分别做什么 42
10.2 QAC：最简单的动作价值 Actor–Critic 42
10.3 Baseline invariance：改变方差，不改变期望梯度 42
10.4 A2C：用优势函数更新 actor 43
10.5 Off-policy Actor–Critic：用重要性采样校正动作分布 43
10.6 Deterministic Actor–Critic 44

11 L1–L10 一页串联 46

内容核对说明 49

统一记号与阅读方法

最常用的记号

记号	名称	一句话解释
S_t, A_t, R_{t+1}	状态、动作、奖励	时刻 t 看见 S_t，选择 A_t，环境随后给出 R_{t+1} 和 S_{t+1}。
$\pi(a \mid s)$	策略	在状态 s 选择动作 a 的概率。
$p(s', r \mid s, a)$	环境模型	执行 a 后同时得到下一状态 s' 和奖励 r 的概率。
γ	折扣因子	控制未来奖励在当前看来有多重要，通常 $0 \le \gamma < 1$。
G_t	回报	从时刻 t 往后收到的折扣奖励总和。
$v_\pi(s), q_\pi(s, a)$	策略 π 的价值	分别评价“从状态开始”和“从状态–动作对开始”有多好。
$v_*(s), q_*(s, a)$	最优价值	所有策略能达到的最好价值。

读公式的固定顺序

看到任何 Bellman 公式时，不要先背求和符号。按下面三步读：

1. **当前先拿到什么？**——即时奖励 R_{t+1}；
2. **之后还剩什么？**——从下一状态开始的价值；
3. **谁决定动作？**——固定策略时由 π 加权，求最优时对动作取最大值。

全书最重要的一句话

$$\boxed{\text{当前价值} = \text{即时奖励} + \gamma \times \text{下一状态的价值}}$$

L2、L3、L4 的各种公式，都是这句话在不同问题下的具体写法。

1　L1：基本概念——把交互问题写成 MDP

本章要回答的问题

智能体面对环境时，什么是状态、动作、策略、奖励和回报？怎样用一个统一的数学对象描述这些元素？

1.1　智能体与环境如何交互

强化学习不是一次性预测，而是一个连续反馈过程：

$$S_t \xrightarrow[\text{智能体按策略选择}]{A_t} (R_{t+1}, S_{t+1}) \xrightarrow{\text{继续决策}} \cdots$$

智能体看见当前状态 S_t，选择动作 A_t；环境响应这个动作，给出奖励 R_{t+1} 和下一状态 S_{t+1}。智能体的目标不是让某一步的奖励最大，而是让**长期累计回报**尽可能大。

1.2　五个基础概念

1.2.1　状态：对决策有用的环境信息

状态 $s \in \mathcal{S}$ 是智能体用来决策的环境描述。在网格世界中，智能体所在的格子可以是状态；在机器人任务中，位置、速度、传感器读数都可能属于状态。

易错点

“状态”不只是随便记录一些观测。理想的状态应包含预测未来所需的信息。如果同一份当前描述对应着不同的未来规律，说明它可能遗漏了历史信息，此时直接把它当作 MDP 状态会有问题。

1.2.2　动作：智能体可以作出的选择

动作集合记作 $\mathcal{A}(s)$。不同状态允许的动作可以不同。例如边界处不能继续向外走；也可以把“向外走”保留为动作，但规定智能体留在原处并受到惩罚。两种建模方式都可以，关键是把规则写清楚。

1.2.3　策略：从状态到动作分布的决策规则

策略写成

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s).$$

如果每个状态只选择一个确定动作，它是**确定性策略**；如果在多个动作之间按概率选择，它是**随机策略**。对每个状态都必须满足

$$\pi(a \mid s) \geq 0, \qquad \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1.$$

1.2.4　环境模型：动作会把世界变成什么样

最一般的单步模型可写为

$$p(s', r \mid s, a) = \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

它同时描述下一状态和奖励的不确定性。确定性网格世界只是特殊情况：给定 s, a 后，某个 s', r 的概率为 1，其余为 0。

课件也分别使用状态转移概率 $p(s' \mid s, a)$ 与奖励概率 $p(r \mid s, a)$。计算期望时可以使用这两个边缘分布；联合形式 $p(s', r \mid s, a)$ 则能保留二者之间可能存在的相关性。

1.2.5　奖励：环境给出的即时反馈

奖励是一步交互后的标量反馈。它告诉智能体“发生了什么结果”，而不直接告诉智能体“正确动作是哪一个”。奖励设计实际上定义了任务目标：

- 到达目标给正奖励；
- 撞墙或进入危险区域给负奖励；
- 普通移动给零奖励或很小的步长代价。

奖励不是价值

奖励 R_{t+1} 只评价刚完成的一步；价值评价从现在起的整个未来。一个动作眼前奖励较小，却可能通向长期回报更高的状态。

1.3　轨迹、回合与回报

1.3.1　轨迹与回合

一条轨迹是交互序列

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots$$

如果任务在终止状态结束，这条有限轨迹称为一个**回合（episode）**，任务称为回合式任务；如果交互没有自然终点，则称为持续式任务。

终止状态之后通常不再产生奖励，可等价地令终止状态的后续价值为 0；也可以增加一个零奖励的吸收状态。两种写法表达的是同一件事。

1.3.2　为什么要有折扣回报

若只把所有奖励直接相加，无穷轨迹的和可能发散。与此同时，我们通常也希望越晚得到的奖励对现在的影响越弱。因此定义

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

这里第一个奖励没有折扣，晚一步的奖励多乘一个 γ：

- γ 小：更看重眼前，策略更“短视”；
- γ 接近 1：更重视长期结果，但价值传播和算法收敛通常更慢；
- $\gamma < 1$ 且奖励有界时，无穷和是有限的。

1.4　Markov 性与 MDP

先抓直觉

如果已经知道当前状态和当前动作，那么预测下一步时不应再需要完整历史。也就是说，当前状态应当是过去信息对未来的充分摘要。

这一点称为 Markov 性：

$$\mathbb{P}(S_{t+1}, R_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, \ldots) = \mathbb{P}(S_{t+1}, R_{t+1} \mid S_t, A_t).$$

若把状态转移与奖励分别描述，一个 MDP 通常写成五元组

$$\boxed{(\mathcal{S}, \mathcal{A}, P, R, \gamma)}.$$

其中 $\mathcal{S}$ 是状态集合，$\mathcal{A}$ 是动作集合，P 是状态转移规律，R 是奖励规律，γ 是折扣因子。也可以用前面的联合分布 $p(s', r \mid s, a)$ 把 P 与 R 合在一起描述。策略 π 不是环境本身的一部分，而是智能体要选择或学习的对象。

关于“奖励整体平移”的适用条件

在无限时域折扣任务中，把所有奖励变成 $ar + b$（其中 $a > 0$）不会改变最优策略，因为每条无限轨迹都增加同样的 $b/(1-\gamma)$，且正比例缩放不改变大小关系。但在长度不同的回合式任务中，每走一步都加 b 会偏好更长或更短的回合，因此不能不加条件地说“奖励平移永远不改变策略”。

本章速记

- MDP 描述环境，策略描述智能体怎么行动。
- 奖励是一时的反馈，回报是未来奖励之和。
- 折扣因子同时影响远期偏好、回报是否有限以及算法收敛速度。
- Markov 性要求当前状态足以预测下一步。
- 到 L2 后，真正被计算和比较的是**期望回报，也就是价值**。

2　L2：状态价值与 Bellman 期望方程

本章要回答的问题

给定一个策略 π，怎样评价它在每个状态有多好？为什么价值之间会互相依赖，又怎样把它们计算出来？

2.1　从一次回报到平均表现

随机策略和随机环境会让同一个起点产生不同轨迹，因此一次实际回报 G_t 不能稳定代表策略质量。更合理的评价是：从该状态出发，把所有可能未来按发生概率取平均。

2.1.1　状态价值

$$\boxed{v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]}$$

下标 π 提醒我们：改变策略后，即使环境没变，状态价值也可能改变。状态价值回答的是：“身处 s，以后一直按 π 行动，平均能得到多少回报？”

2.1.2　动作价值

$$\boxed{q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]}$$

它回答：“在 s 先强制做动作 a，以后再按 π 行动，平均能得到多少回报？”正因为它分别评价每个动作，所以后面可以用它改进策略。

状态价值与动作价值的关系

在状态 s，策略会以概率 $\pi(a \mid s)$ 选择各个动作。因此，状态价值就是这些动作价值的加权平均：

$$\boxed{v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) q_\pi(s,a)}.$$

2.2　Bellman 方程为什么会出现

2.2.1　第一步：把回报拆成“现在 + 未来”

从回报定义出发：

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \\ &= R_{t+1} + \gamma\big(R_{t+2} + \gamma R_{t+3} + \cdots\big) \\ &= \boxed{R_{t+1} + \gamma G_{t+1}}. \end{aligned}$$

最后一行不是额外假设，只是把同一个无穷和重新分组。

2.2.2　第二步：在固定策略下取条件期望

站在状态 s：

1. 策略以 $\pi(a \mid s)$ 的概率选择动作 a；
2. 环境以 $p(s', r \mid s, a)$ 的概率给出奖励 r 和下一状态 s'；
3. 从 s' 开始，剩余期望回报就是 $v_\pi(s')$。

把所有可能的 a, s', r 按概率加权，得到

$$\boxed{v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma v_\pi(s') \right]}.$$

这就是 **Bellman 期望方程**。它不是一个额外的学习规则，而是价值定义在 Markov 性下必然满足的自洽关系。

2.3　动作价值的 Bellman 形式

若第一步动作 a 已经给定，就不再需要对当前动作按 π 求平均：

$$\boxed{q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma v_\pi(s') \right]}.$$

再代入 $v_\pi(s') = \sum_{a'} \pi(a' \mid s') q_\pi(s', a')$，也可以得到只含 q_π 的递推式。记忆时不必死背展开形式，只需记住：**第一步固定为 a，下一步起重新服从 π**。

2.4　一个两状态例子

考虑固定策略下的确定性过程：

- 从 s_1 得到奖励 0，然后进入 s_2；
- 在 s_2 每一步得到奖励 1，并留在 s_2。

先从容易的 s_2 看。它现在得到 1，下一步又回到同一个状态，所以

$$v_\pi(s_2) = 1 + \gamma v_\pi(s_2).$$

移项后得到

$$v_\pi(s_2) = \frac{1}{1 - \gamma}.$$

而 s_1 当前奖励为 0，随后进入 s_2，因此

$$v_\pi(s_1) = 0 + \gamma v_\pi(s_2) = \frac{\gamma}{1 - \gamma}.$$

这个例子把 Bellman 方程的含义展示得很清楚：**一个状态的价值，可以借用下一状态的价值来表达**。这种“用已有估计更新另一个估计”的思想称为 **bootstrapping**。

2.5　怎样求出所有状态价值

2.5.1　矩阵形式

对固定策略，把每个状态的平均即时奖励放进向量 $\boldsymbol{r}_\pi$，把策略诱导出的状态转移概率放进矩阵 P_π，所有状态价值放进 $\boldsymbol{v}_\pi$。各状态的 Bellman 方程可一起写成

$$\boxed{\boldsymbol{v}_\pi = \boldsymbol{r}_\pi + \gamma P_\pi \boldsymbol{v}_\pi}.$$

如果状态数有限且 $\gamma < 1$，则 $I - \gamma P_\pi$ 可逆，于是

$$\boldsymbol{v}_\pi = (I - \gamma P_\pi)^{-1} \boldsymbol{r}_\pi.$$

这是理论上的闭式表达，但大规模问题通常不显式求逆。

2.5.2　迭代策略评价

Bellman 方程本身已经暗示了一种迭代方法。先任取初值 $\boldsymbol{v}_0$，然后反复执行

$$\boxed{\boldsymbol{v}_{k+1} = \boldsymbol{r}_\pi + \gamma P_\pi \boldsymbol{v}_k}.$$

每轮都用旧价值计算“一步奖励 + 折扣后的下一状态价值”。在本文假设下，无论初值如何，这个序列都会收敛到真正的 $\boldsymbol{v}_\pi$。

下标最容易混淆的地方

v_π 中的 π 表示“这是谁的价值”；v_k 中的 k 表示“当前是第几次近似”。迭代尚未收敛时，v_k 一般还不等于任何策略的精确价值。

本章速记

- 固定策略评价的对象是 v_π 和 q_π，二者都是期望回报。
- Bellman 期望方程来自 $G_t = R_{t+1} + \gamma G_{t+1}$。
- 固定策略时，对动作按 $\pi(a \mid s)$ 加权，而不是取最大。
- v_π 是当前策略各动作价值的加权平均。
- 求出 v_π 叫作**策略评价**；它为下一步策略改进提供依据。

3　L3：最优策略与 Bellman 最优方程

本章要回答的问题

怎样严格定义“更好的策略”？最优价值满足什么方程？为什么解这个方程就能得到最优策略？

3.1　从评价转向控制

L2 的问题是：**给定策略，算它有多好**。 L3 的问题变为：**在所有策略中，找一个最好的**。

若对所有状态都有

$$v_{\pi_1}(s) \geq v_{\pi_2}(s),$$

就说 π_1 不劣于 π_2。最优状态价值定义为

$$\boxed{v_*(s) = \max_{\pi} v_\pi(s)}.$$

最优动作价值定义为

$$\boxed{q_*(s,a) = \max_{\pi} q_\pi(s,a)}.$$

在有限折扣 MDP 中，至少存在一个平稳、确定性的最优策略。最优策略不一定唯一：如果多个动作并列最优，它们都可以被选；只在这些并列最优动作上随机化，也仍然是最优策略。

3.2　Bellman 最优方程怎样得到

先抓直觉

站在状态 s，最优决策可以分成两部分：**当前先选一个最好的动作**，然后从下一状态开始继续最优地行动。因此“固定策略下的平均”会被“对动作取最大”替代。

若当前选择动作 a，一步之后的最佳期望回报为

$$\sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right].$$

这就是最优动作价值：

$$\boxed{q_*(s,a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right]}.$$

当前状态应该选价值最大的动作，于是

$$\boxed{v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s,a)}.$$

把两式合并，得到 **Bellman 最优方程**：

$$\boxed{v_*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right]}.$$

3.3　期望方程与最优方程不要混

	Bellman 期望方程	Bellman 最优方程
回答的问题	给定 π，它的价值是多少?	所有策略中最好的价值是多少?
动作如何处理	按 $\pi(a \mid s)$ 加权平均	对 a 取最大值
未知量	v_π 或 q_π	v_* 或 q_*
方程性质	固定 π 后是线性的	含有 max，通常是非线性的
对应任务	策略评价	最优控制

3.4　为什么最优方程有唯一解

定义 Bellman 最优算子

$$(\mathcal{T}_* v)(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v(s') \right].$$

Bellman 最优方程就是固定点方程

$$v = \mathcal{T}_* v.$$

“压缩”是什么意思？

假设两个价值猜测 v, w 最多相差 d。经过一次 Bellman 更新后，即时奖励部分相同，差异只来自下一状态价值，而这一部分还要乘 γ。因此新的最大差异至多是 γd。

用无穷范数 $\|v\|_\infty = \max_s |v(s)|$ 表示“所有状态中的最大误差”，可以严格写成

$$\boxed{\|\mathcal{T}_* v - \mathcal{T}_* w\|_\infty \le \gamma \|v - w\|_\infty}.$$

因为 $\gamma < 1$，每次更新都会把最坏误差按比例缩小。压缩映射定理因此给出三个重要结论：

1. **存在性：** Bellman 最优方程至少有一个解；
2. **唯一性：** 这个解只有一个，也就是 v_*；
3. **可计算性：** 从任意 v_0 开始反复做 $v_{k+1} = \mathcal{T}_* v_k$，都会收敛到 v_*。

范数条件

Bellman 最优算子标准地是在**无穷范数**下具有折扣因子 γ 的压缩性。不能把它无条件扩展为“在任意向量范数下都是 γ-压缩”。

3.5　从最优价值取出最优策略

一旦知道 v_*，就在每个状态计算“一步奖励 + 最优后续价值”，并选择最大的动作：

$$a_*(s) \in \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right].$$

令

$$\pi_*(a \mid s) = \begin{cases} 1, & a = a_*(s), \\ 0, & a \neq a_*(s), \end{cases}$$

便得到一个确定性最优策略。这叫对 v_* **贪心**。

“贪心”并不等于只看即时奖励

这里比较的是 $r + \gamma v_*(s')$，它已经包含未来的最优价值；所以这是对长期回报贪心，而不是只挑眼前奖励最大的动作。

3.6　什么会改变最优策略

- **环境模型：**相同动作若通向不同状态，最优选择当然会变；
- **奖励设计：**更强的危险惩罚会让策略更保守；
- **折扣因子：**γ 越小越重眼前，可能不愿为远期目标绕路；
- **正比例缩放：**在通常条件下，$r \mapsto ar$、$a > 0$ 不改变大小关系，因此不改变最优策略；
- **常数平移：**对无限时域折扣任务不改变最优策略，但对长度可变的回合式任务需要谨慎。

本章速记

- 固定策略：对动作求平均；追求最优：对动作取最大。
- $v_*(s) = \max_a q_*(s, a)$。
- Bellman 最优算子在无穷范数下是 γ-压缩映射。
- 因而最优价值存在、唯一，而且可以从任意初值迭代求出。
- 对最优价值做一步贪心，就能得到一个最优策略。

4　L4：价值迭代、策略迭代与截断策略迭代

本章要回答的问题

L3 已经知道最优价值满足什么方程。现在怎样真正把它算出来？价值迭代和策略迭代有何区别，又有什么共同结构？

4.1　先明确：这是有模型的动态规划

L4 的算法需要知道 $p(s', r \mid s, a)$，因此属于**动态规划（DP）**和**基于模型**的方法。后面的 Monte Carlo 与 TD 方法会讨论模型未知时，怎样利用采样到的经验代替模型中的期望。

4.2　价值迭代（Value Iteration, VI）

4.2.1　算法从哪里来

L3 已知 Bellman 最优方程是固定点方程 $v_* = \mathcal{T}_* v_*$，而压缩映射定理告诉我们可以从任意 v_0 反复应用 $\mathcal{T}_*$。因此

$$\boxed{v_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_k(s') \right]}.$$

为了更容易读，可以把一轮拆成两步。先对每个动作做一步展望：

$$q_k(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_k(s') \right],$$

再选最大者：

$$v_{k+1}(s) = \max_a q_k(s, a).$$

4.2.2　初学者版伪代码

价值迭代的一轮

1. 任意初始化每个状态的数值 $v_0(s)$；
2. 对每个状态 s，用旧表 v_k 计算每个动作的 $q_k(s, a)$；
3. 把最大动作价值写入新表：$v_{k+1}(s) = \max_a q_k(s, a)$；
4. 重复，直到相邻两轮的最大变化足够小；
5. 最后对收敛后的价值表做贪心，取出策略。

易错点

中间的 v_k 只是对 v_* 的近似，一般不必是某个策略的精确状态价值。价值迭代的核心是直接逼近 Bellman 最优方程的固定点。

4.3　策略迭代（Policy Iteration, PI）

策略迭代显式维护一个策略，并反复进行**策略评价**与**策略改进**。

4.3.1　第一步：策略评价

给定当前策略 π_k，求它的精确价值：

$$\boxed{v_{\pi_k}(s) = \sum_a \pi_k(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_{\pi_k}(s') \right]}.$$

这就是 L2 的 Bellman 期望方程。可通过解线性方程组，或反复应用 $\mathcal{T}_\pi$ 直到收敛来完成。

4.3.2　第二步：策略改进

利用刚算出的 v_{π_k}，对每个状态做一步展望：

$$q_{\pi_k}(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_{\pi_k}(s') \right].$$

然后选取动作价值最大的动作

$$\boxed{a_{k+1}(s) \in \arg\max_a q_{\pi_k}(s, a)},$$

并令新策略在状态 s 确定地选择 $a_{k+1}(s)$。

为什么改进后不会更差？

$v_{\pi_k}(s)$ 是按旧策略选择动作所得 $q_{\pi_k}(s, a)$ 的平均值。最大值不会小于平均值，所以在每个状态改选最大动作，至少不会让“一步之后继续按旧策略”的回报变差。策略改进定理进一步保证

$$v_{\pi_{k+1}}(s) \geq v_{\pi_k}(s), \qquad \forall s.$$

4.3.3　何时停止

如果贪心改进后策略不再变化，当前策略满足对自身价值贪心，它就是最优策略。在有限 MDP 中，配合一致的并列动作处理方式，策略迭代会在有限次策略改进后停在某个最优策略。

并列动作

多个动作价值相同时，若实现中每轮随意切换并列动作，可能出现“策略看起来一直在变、价值却没有变”的现象。实际编程时应使用固定的 tie-breaking 规则，或把“只在并列动作间变化”也视为已经稳定。

4.4　截断策略迭代（Modified/Truncated PI）

精确策略评价可能很贵。一个自然的折中是：每次只做有限轮策略评价，就提前进行策略改进。这通常称为**截断策略迭代**或**修正策略迭代**。

设每次策略评价只执行 m 次：

- $m = 1$：只做一次 Bellman 更新，行为接近价值迭代；
- m 较大：更充分地评价当前策略；
- $m \to \infty$：得到精确策略评价，对应标准策略迭代。

因此 VI 与 PI 不是毫无关系的两套算法，而是“评价深度”这条轴线上的两个端点。

4.5　三种算法的对比

	价值迭代	截断策略迭代	策略迭代
维护的核心对象	价值近似 v_k	策略与近似评价值	策略与精确评价值
每次评价深度	一步	固定或自适应的有限步	直到评价收敛或精确求解
单轮开销	通常较小	居中	通常较大
外层轮数	可能较多	居中	常较少
中间价值是否对应某策略	一般不对应精确价值	近似对应当前策略	是当前策略的精确价值
共同前提	知道环境模型；都利用 Bellman 更新与贪心改进。		

4.6　实现时最值得记住的四点

1. **同步更新与原地更新要分清**。公式通常用完整的旧表 v_k 生成新表 v_{k+1}；原地更新会立刻使用本轮刚得到的新值，通常也能工作，但更新顺序会影响过程。
2. **用最大变化监测收敛**。常见指标是 $\Delta_k = \max_s |v_{k+1}(s) - v_k(s)|$。它足够小时停止，但阈值越小，计算越久。
3. **γ 越接近 1，通常收敛越慢**。因为压缩比例就是 γ，误差每轮缩小得更少。
4. **模型未知时不能直接照搬**。求和中的转移概率和奖励规律未知时，需要用样本估计；这正是后续 MC、TD 和 Q-learning 等方法要解决的问题。

本章速记

- VI：直接反复应用 Bellman 最优算子，逼近 v_*。
- PI：精确评价当前策略，再对它的价值做贪心改进。
- 截断 PI：只评价有限步，是 VI 与 PI 之间的折中。
- 三者都需要模型，都是“展望一步 + 利用后续价值”的动态规划。
- 评价阶段回答“当前策略多好”，改进阶段回答“有没有更好的动作”。

5　L5：Monte Carlo 方法——没有模型时，用完整经历估计价值

本章要回答的问题

L4 的 VI/PI 都要知道环境模型 $p(s', r \mid s, a)$。如果模型不知道，还能不能做策略评价和策略改进？Monte Carlo 方法的答案是：可以，不直接算模型里的期望，而是用真实或模拟出来的 episode 回报做样本平均。

5.1　从“知道模型”到“只拿到样本”

在 L2–L4 中，很多更新都长这样：

$$\sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v(s') \right].$$

这表示：如果我们知道环境的转移概率和奖励分布，就能把所有可能的下一步结果按概率加权平均。问题是，真实任务里往往不知道这个模型。

Monte Carlo（MC）方法换了一个角度：既然价值本来就是**回报的期望**，那就直接多试几次，把实际观察到的回报取平均。

先抓直觉

模型方法像是在看“环境说明书”：知道每种结果的概率，于是直接算期望。MC 方法像是在做实验：不知道说明书，但可以反复采样，用大量实验结果的平均值逼近期望。

对动作价值来说，我们真正想估计的是

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

这句话的意思是：从状态 s 出发，第一步固定做动作 a，之后按照策略 π 行动，最终能得到多少折扣回报的平均值。

如果我们从同一个 (s, a) 出发，按 π 生成很多条 episode，每条 episode 都给出一个实际回报 $G^{(i)}(s, a)$。于是可以用

$$\boxed{\hat{q}_\pi(s, a) = \frac{1}{N} \sum_{i=1}^{N} G^{(i)}(s, a)}$$

来近似 $q_\pi(s, a)$。这里的帽子 $\hat{q}$ 提醒我们：它不是精确值，而是由样本得到的估计。

易错点

课件用均值估计引入 MC 时强调了 iid 样本的大数定律。在 RL 中，若多条 episode 是在相同策略下独立生成的，对应回报可以较自然地看成同分布样本；但同一条 episode 内不同访问位置通常相关。实际算法仍常用这些样本，直觉仍是“样本越多，均值越稳”，只是严格分析会比普通 iid 均值更细。

5.2　MC Basic：把策略迭代改成无模型版本

L4 的策略迭代有两步：**先评价当前策略，再根据价值改进策略**。MC Basic 保留这个框架，只把“评价”这一步从解 Bellman 方程改成采样估计。

MC Basic 的一轮

1. 固定当前策略 π_k；
2. 对每个状态–动作对 (s,a)，从这里开始采集足够多条 episode；
3. 每条 episode 后续都按 π_k 行动，记录它的回报；
4. 用这些回报的平均值估计 $q_{\pi_k}(s,a)$；
5. 对每个状态选择估计动作价值最大的动作，得到新策略 π_{k+1}。

策略改进时不再需要环境模型，只需要比较当前估计表：

$$a_k^*(s) \in \arg\max_a \hat{q}_k(s,a).$$

然后令新策略在状态 s 选择这个动作。

为什么 MC 更常估计 q 而不是 v？

如果模型已知，只知道 $v(s')$ 也能通过一步展望算出每个动作的好坏。但模型未知时，无法可靠计算 $\sum_{s',r} p(s',r \mid s,a)[r+\gamma v(s')]$。所以无模型控制通常直接估计动作价值 $q(s,a)$，因为它可以直接告诉我们“在状态 s 选哪个动作更好”。

5.3　更有效地使用一条 episode

MC Basic 的想法很清楚，但效率不高：如果每个 (s,a) 都单独生成很多 episode，采样成本会很大。实际上，一条 episode 里可能出现许多状态–动作对，每次出现都携带了从那一刻开始的后续回报信息。

设一条 episode 中从时间 t 开始的回报为

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots.$$

实际计算时常从 episode 末尾往前扫，用递推形式更新：

$$G \leftarrow R_{t+1} + \gamma G.$$

这样每扫到一个访问到的 (S_t, A_t)，就能得到一个对应的回报样本。

常见的使用方式有两种：

- **first-visit MC：**同一条 episode 中，一个 (s,a) 若出现多次，只用第一次访问后的回报；
- **every-visit MC：**每次访问 (s,a) 都使用，因此一条 episode 贡献更多样本。

课件中的 MC Exploring Starts 正是把这些想法合在一起：每产生一条 episode，就从后往前计算回报，并更新访问过的动作价值。这比“等某个 (s,a) 的所有 episode 收集完再更新”更像后续 RL 算法的工作方式。

广义策略迭代

很多 RL 算法都不再严格区分“先完全评价，再完全改进”。它们通常是一边让价值估计变好，一边让策略变好。这种评价与改进交替推进的思想，称为 generalized policy iteration（GPI）。

5.4　Exploring Starts：理论上好用，实践中难满足

MC 控制要想找到好策略，必须让每个动作都有机会被尝试。否则某个从未被试过的动作可能其实很好，但算法永远不知道。

Exploring starts 要求 episode 的初始状态–动作对可以覆盖所有 (s,a)，并且每一对都有机会被选为起点。这样做的好处是理论上干净：每个动作价值都能被充分估计。坏处也很明显：真实系统通常不能随意指定“从任意状态、任意动作开始”。

例如机器人、游戏或真实控制任务中，当前状态是环境演化出来的，不是我们想把它摆到哪里就摆到哪里。

5.5　ε-greedy：用软策略替代 exploring starts

为了不依赖 exploring starts，课件引入了**软策略（soft policy）**：在每个状态下，每个动作都有正概率被选择。最常见的软策略是 ε-greedy。

先找到当前估计下的贪心动作：

$$a^*(s) \in \arg\max_a \hat{q}(s,a).$$

然后大部分时间选它，小部分时间随机探索其他动作：

$$\pi(a \mid s) = \begin{cases} 1-\varepsilon+\dfrac{\varepsilon}{|\mathcal{A}(s)|}, & a = a^*(s), \\ \dfrac{\varepsilon}{|\mathcal{A}(s)|}, & a \neq a^*(s). \end{cases}$$

这里 $|\mathcal{A}(s)|$ 是状态 s 下可选动作的数量。可以把它理解为：先拿出 ε 的概率做均匀随机探索，剩下 $1-\varepsilon$ 的概率交给当前最优动作。

探索与最优性不能同时免费获得

ε 大，探索能力强，但会经常故意不选当前最好的动作；ε 小，更接近贪心策略，但可能探索不足。固定 $\varepsilon > 0$ 时，算法通常是在固定的 ε-soft 策略集合中改进，不等于已经得到原问题的绝对最优确定性策略。实践中常使用逐渐减小的 ε，让早期多探索、后期更贪心。

5.6　L5 与前四讲的关系

问题	L4 的做法	L5 的做法
如何评价策略	用模型写出 Bellman 方程并求解	用回报样本均值估计 $q_\pi(s,a)$
如何改进策略	对模型计算出的动作价值贪心	对采样估计出的动作价值贪心或 ε-greedy
是否需要模型	需要 $p(s',r \mid s,a)$	不需要模型，但需要能采样
主要代价	计算模型期望或解方程	采样量大，且要保证足够探索

本章速记

- MC 的核心：价值是回报的期望，期望可以用样本平均估计。
- 无模型控制通常直接估计 $q(s,a)$，因为它能直接比较动作。
- MC Basic 是“策略迭代的无模型版本”：采样评价 q，再贪心改进。
- first-visit/every-visit 是更充分使用 episode 数据的方式。
- exploring starts 保证每个 (s,a) 被探索，但实践中很难满足。
- ε-greedy 用持续随机探索替代随意指定起点。

6　L6：随机逼近——为什么样本更新可以逐渐靠谱

本章要回答的问题

L5 用样本平均估计价值，但实际算法通常不会等所有样本都收集完再统一计算。它们更常见的形式是：来一个样本，就把当前估计往样本方向推一点。L6 解释这种随机、增量式更新背后的数学思想：随机逼近、Robbins–Monro 算法和随机梯度下降。

6.1　从普通平均到增量平均

假设我们想估计随机变量 X 的均值 $\mathbb{E}[X]$。最直接的办法是先收集 N 个样本，再计算

$$\bar{x}_N = \frac{1}{N}\sum_{i=1}^{N} x_i.$$

但在 RL 中，样本通常是不断到来的。我们希望每来一个样本就更新一次估计，而不是等到最后。

设 w_k 表示当前对均值的估计。新样本 x_k 到来时，一种非常重要的更新形式是

$$\boxed{w_{k+1} = w_k + \alpha_k(x_k - w_k)}$$

其中 $\alpha_k > 0$ 是步长。

这条式子最好这样读：

- $x_k - w_k$：新样本和当前估计之间的差，表示“这次样本认为你偏了多少”；
- α_k：相信这次样本多少；
- w_{k+1}：把旧估计朝新样本方向移动一小步。

当 $\alpha_k = 1/k$ 时，这个递推正好等价于普通样本平均。当 α_k 取其他满足条件的序列时，它仍可能收敛，这就进入了随机逼近的范围。

先抓直觉

后面很多 RL 更新都长得像

$$\text{新估计} = \text{旧估计} + \alpha \times (\text{目标} - \text{旧估计}).$$

L6 的作用，就是先用均值估计和 SGD 解释这个模板为什么有道理。

6.2　Robbins–Monro：只看到带噪声的函数值，也能找根

随机逼近（stochastic approximation, SA）研究的是：目标函数或方程本身不知道精确表达式，只能通过带噪声的数据观察它，是否仍能通过迭代逼近正确答案。

Robbins–Monro（RM）算法考虑一个找根问题：

$$g(w) = 0.$$

如果 g 完全已知，可以用普通数值方法找根。但现在我们只能在 w_k 处观测到一个带噪声的值

$$\tilde{g}(w_k) = g(w_k) + \eta_k.$$

RM 的更新是

$$\boxed{w_{k+1} = w_k - \alpha_k \tilde{g}(w_k)}$$

也就是：把带噪声的观测值当作方向信号，用小步长不断修正当前估计。

为什么是减号？

在一维、g 单调递增的情况下，若当前 w_k 在根的右边，通常有 $g(w_k) > 0$，减去它会把 w_k 往左推；若 w_k 在根的左边，通常有 $g(w_k) < 0$，减去负数会把 w_k 往右推。噪声会让每一步不完美，但只要步长安排得好，长期趋势仍可能指向根。

6.3　步长条件：既要走得远，也要越来越稳

课件中的 RM 收敛定理可以先不记完整技术细节，但一定要记住三类条件：

1. **函数形状要合适**。在一维版本中，常要求 g 单调并且斜率有上下界，这样根不会乱跑，也不会太平或太陡。
2. **步长要“无限但平方有限”**。常见条件是

$$\sum_{k=1}^{\infty} \alpha_k = \infty, \qquad \sum_{k=1}^{\infty} \alpha_k^2 < \infty.$$

   第一条保证总共走得足够远，不会太早停住；第二条保证后期步子足够小，噪声影响会被逐渐压下去。

3. **噪声不能系统性误导**。通常要求噪声在给定过去信息时均值为 0，并且方差有限。

典型选择是 $\alpha_k = 1/k$。它满足上面两条步长条件：调和级数发散，而平方和收敛。

易错点

随机逼近的收敛不是“只要用了样本更新就一定收敛”。步长、噪声、样本生成方式、目标函数形状都会影响结论。这点在后面看 TD、Q-learning、函数逼近时尤其重要。

6.4　均值估计其实是 RM 的特例

现在回到最熟悉的均值估计。我们想找到 $w = \mathbb{E}[X]$。这可以写成一个找根问题：

$$g(w) = w - \mathbb{E}[X] = 0.$$

但 $\mathbb{E}[X]$ 不知道，我们只能看到样本 x_k。于是用

$$\tilde{g}(w_k) = w_k - x_k$$

作为 $g(w_k)$ 的带噪声观测。代入 RM 更新：

$$w_{k+1} = w_k - \alpha_k(w_k - x_k) = w_k + \alpha_k(x_k - w_k).$$

这正是前面的增量均值更新。

所以，L6 的一个核心观点是：**样本均值不是孤立技巧，而是随机逼近理论中的一个简单例子**。这也解释了为什么 L5 中“用样本回报估计价值”的做法可以和迭代更新自然结合。

6.5　SGD：用一个样本梯度近似真实梯度

随机梯度下降（stochastic gradient descent, SGD）处理的是优化问题：

$$\min_w J(w) = \mathbb{E}[f(w, X)].$$

如果知道真实梯度，就可以做普通梯度下降：

$$w_{k+1} = w_k - \alpha_k \nabla J(w_k).$$

但真实梯度通常也是一个期望：

$$\nabla J(w_k) = \mathbb{E}[\nabla_w f(w_k, X)].$$

当这个期望难以精确计算时，SGD 用一个样本 x_k 的梯度来替代它：

$$\boxed{w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k)}.$$

SGD 为什么也能放进 RM 框架？因为最优化的一阶条件是

$$\nabla J(w) = 0.$$

也就是说，优化可以看作“寻找梯度为零的点”。样本梯度 $\nabla_w f(w_k, x_k)$ 就是对真实梯度 $\nabla J(w_k)$ 的带噪声观测。

均值估计也是 SGD

若取

$$f(w, X) = \frac{1}{2}\|w - X\|^2,$$

则样本梯度是 $w - X$。SGD 更新变成

$$w_{k+1} = w_k - \alpha_k(w_k - x_k) = w_k + \alpha_k(x_k - w_k),$$

又回到了增量均值估计。

6.6　BGD、MBGD 与 SGD 的区别

三者的区别主要在于：每次更新时用多少样本估计梯度。

方法	每轮用的样本	优点	代价
BGD	全部样本	梯度估计最稳定	每轮计算贵，数据大时不灵活
SGD	一个样本	每轮便宜，能在线更新	随机性大，轨迹会抖动
MBGD	一小批样本	在稳定性和效率之间折中	需要选择 batch 大小

SGD 的典型现象是：离最优点很远时，真实梯度信号通常较强，随机误差的相对影响较小，所以能较快靠近正确区域；离最优点很近时，真实梯度变小，样本噪声显得更明显，于是更新轨迹会在附近抖动。递减步长的作用，就是让这种后期抖动逐渐变小。

6.7　L6 对后续 RL 的意义

L6 不只是单独讲优化。它是在为后面的 TD 学习、Q-learning、actor–critic 等算法铺垫一种反复出现的模板：

$$\text{参数或价值的新估计} = \text{旧估计} + \alpha_k\,(\text{带噪声的目标信号} - \text{旧估计}).$$

在 L5 中，目标信号来自一整条 episode 的实际回报；在 TD 中，目标信号会变成“一步奖励 $+\gamma$ 下一状态估计价值”。因此，理解 L6 能帮助你看懂后面算法为什么看起来都像在做“误差修正”。

本章速记

- 增量均值更新：旧估计朝新样本方向移动一小步。
- RM 算法：只观察到带噪声的 $g(w)$，仍尝试求 $g(w) = 0$。
- 步长要既不太快消失，也要最终足够小：$\sum \alpha_k = \infty$，$\sum \alpha_k^2 < \infty$。
- 均值估计是 RM 的特例，也是 SGD 的特例。
- SGD 用样本梯度替代真实期望梯度，因此每步便宜但带随机性。
- L6 的思想会在 TD、Q-learning 等后续算法中反复出现。

7　L7：时序差分学习——不等 episode 结束就更新

本章要回答的问题

MC 必须等一条 episode 结束后，才能用完整回报更新价值。能否只观察一步转移，就立刻更新？怎样从状态价值过渡到 Sarsa、n-step Sarsa 和 Q-learning？为什么 Sarsa 是 on-policy，而 Q-learning 可以 off-policy?

7.1　先看全章地图：L7 是 L2、L5、L6 的交汇点

TD 学习和 MC 一样**不需要环境模型**，但它不再等待完整回报，而是把 Bellman 方程右侧的期望换成当前样本，再按 L6 的随机逼近形式更新：

Bellman 方程给出应满足的固定点

⇓

用一步或多步经验构造带噪声的 TD 目标

⇓

旧估计朝 TD 目标移动一小步

因此，L7 不是突然出现的一组新公式，而是把前面三条线接在了一起：

- **来自 L2：**价值函数满足 Bellman 方程；
- **来自 L5：**模型未知时，用实际经验样本代替模型期望；
- **来自 L6：**用“新估计 = 旧估计 + 步长 × 修正信号”逐步逼近方程的解。

TD 最核心的变化

MC 的目标是从当前时刻一直累加到 episode 结束的真实回报；一步 TD 的目标只看

实际得到的一步奖励 $+\gamma\times$ 自己对下一状态价值的当前估计.

也就是说，TD 用一个尚未完全学准的估计去更新另一个估计。这种做法叫作 **bootstrapping（自举）**。

7.2　基础 TD：用一步样本估计状态价值

7.2.1　从 Bellman 期望方程得到 TD 目标

给定策略 π，目标是估计所有状态的 $v_\pi(s)$。根据 Bellman 期望方程，

$$v_\pi(s)=\mathbb{E}[R_{t+1}+\gamma v_\pi(S_{t+1})\,|\,S_t=s]\,.$$

如果当前在 s_t，并实际观察到一步样本 (s_t,r_{t+1},s_{t+1})，就用

$$\boxed{\bar{v}_t=r_{t+1}+\gamma v_t(s_{t+1})}$$

作为右侧期望的一次样本近似。课本把 $\bar{v}_t$ 称为 **TD target（TD 目标）**。

于是，基础 TD 更新为

$$\boxed{v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\left[v_t(s_t) - \big(r_{t+1} + \gamma v_t(s_{t+1})\big)\right]}$$

并且

$$v_{t+1}(s) = v_t(s), \qquad s \neq s_t.$$

这第二行经常被省略，却很重要：**一次样本只更新本次访问到的状态**，没有访问到的状态保持不变。

7.2.2　把公式读成“旧值朝目标移动”

课本采用的 TD error 记号是

$$\boxed{\delta_t = v_t(s_t) - \bar{v}_t = v_t(s_t) - \big(r_{t+1} + \gamma v_t(s_{t+1})\big)}$$

所以更新式也可写成

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\delta_t.$$

把符号展开，则是更直观的形式：

$$v_{t+1}(s_t) = v_t(s_t) + \alpha_t(s_t)\underbrace{\left[\bar{v}_t - v_t(s_t)\right]}_{\text{目标减当前估计}}.$$

若在这一次更新中暂时把 $\bar{v}_t$ 当作固定目标，且 $0 < \alpha_t(s_t) < 1$，那么

$$v_{t+1}(s_t) - \bar{v}_t = \big(1 - \alpha_t(s_t)\big)\left[v_t(s_t) - \bar{v}_t\right],$$

从而

$$\left|v_{t+1}(s_t) - \bar{v}_t\right| < \left|v_t(s_t) - \bar{v}_t\right|.$$

这正是“target”一词的数学含义：更新后的估计比更新前更靠近本次目标。

TD error 的正负号

本教材把 TD error 定义为“当前估计 − TD 目标”，所以更新写成 $-\alpha_t\delta_t$。读公式时应以本教材的定义为准；最不容易记错的办法，是直接读成

$$\text{新值} = \text{旧值} + \alpha_t(\text{目标} - \text{旧值}).$$

7.2.3　TD error 为什么能反映估计是否准确

当 $v_t = v_\pi$ 时，在给定 $S_t = s_t$ 的条件下，

$$\begin{aligned}\mathbb{E}[\delta_t \mid S_t = s_t] &= v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s_t] \\ &= 0.\end{aligned}$$

第二个等号正是 Bellman 期望方程。因此，TD error 不只是 t 与 $t+1$ 两个时刻估计之间的差；它还是新经验 (s_t, r_{t+1}, s_{t+1}) 带来的 **innovation（新信息）**。TD 学习所做的事，就是用这份新信息修正当前估计。

7.3　TD 与 Monte Carlo：都无模型，但更新时机不同

比较角度	TD / Sarsa	Monte Carlo
何时更新	得到一步经验后即可立即更新，是增量式方法	必须等 episode 完整结束后，才能算折扣回报
适用任务	可用于 episodic task，也可用于没有终止状态的 continuing task	依赖有限长度 episode，适用于 episodic task
是否自举	是；目标中含下一状态或状态–动作对的当前估计	否；直接使用 episode 的实际折扣回报
是否依赖初值	自举需要价值的初始猜测，学习速度会受初值影响	完整回报不依赖下一状态的价值初值
估计方差	只涉及较少的随机变量，方差较低	完整回报包含许多随机奖励和状态，方差较高

不是“TD 永远优于 MC”

表中的差异来自目标的构造方式。TD 只看较短的随机片段，方差低，但会受尚未学准的自举值影响；MC 使用完整实际回报，不依赖自举初值，但整条轨迹带来更多随机性。后面的 n-step Sarsa 正是在这两个端点之间调节。

7.4　基础 TD 的收敛条件

对固定策略 π，课本给出的结论是：如果对每个 $s \in \mathcal{S}$ 都有

$$\sum_t \alpha_t(s) = \infty, \qquad \sum_t \alpha_t^2(s) < \infty,$$

那么基础 TD 的 $v_t(s)$ 几乎必然收敛到 $v_\pi(s)$。

这里 $\alpha_t(s) > 0$ 只发生在状态 s 于时刻 t 被访问时；否则把它看作 0。所以第一条条件还隐含了：**每个需要估计的状态必须被访问无穷多次，或至少足够多次**。这要依靠 exploring starts 或具有足够探索能力的策略。

课本也说明，实践中常用一个很小的常数步长。这时平方和条件不再成立，但仍可讨论期望意义下的收敛。在控制任务中，待评价的策略还会不断变化；常数步长能让价值估计继续跟踪这个非平稳目标，而逐渐过小的步长可能来不及适应新策略。

收敛结论有前提

“TD 会收敛”不是无条件命题。这里的定理针对**给定策略的价值评价**，并要求逐状态满足步长条件和充分访问。把 TD 与策略改进组合成控制算法后，策略本身在变化，不能把固定策略的定理不加区分地直接套用。

7.5　Sarsa：把状态价值 TD 改成动作价值 TD

7.5.1　为什么要从 v_π 换成 q_π

基础 TD 只能估计 $v_\pi(s)$。要直接比较同一状态下不同动作的好坏，需要动作价值 $q_\pi(s,a)$。它满足动作价值形式的 Bellman 方程：

$$\boxed{q_\pi(s,a)=\mathbb{E}[R_{t+1}+\gamma q_\pi(S_{t+1},A_{t+1})\,|\,S_t=s,A_t=a]}.$$

只要把基础 TD 中的“状态”换成“状态–动作对”，就得到 Sarsa。

7.5.2　Sarsa 更新与名字的来源

对按策略 π 产生的经验

$$(s_t,a_t,r_{t+1},s_{t+1},a_{t+1}),$$

Sarsa 的目标和更新分别是

$$\bar{q}_t^{\text{Sarsa}}=r_{t+1}+\gamma q_t(s_{t+1},a_{t+1}),$$

$$\boxed{q_{t+1}(s_t,a_t)=q_t(s_t,a_t)-\alpha_t(s_t,a_t)\left[q_t(s_t,a_t)-\bar{q}_t^{\text{Sarsa}}\right]}.$$

其余 $(s,a)\neq(s_t,a_t)$ 的动作价值保持不变。

算法每次需要的五个量依次是

$$\underbrace{s_t}_{\text{State}}\ \underbrace{a_t}_{\text{Action}}\ \underbrace{r_{t+1}}_{\text{Reward}}\ \underbrace{s_{t+1}}_{\text{State}}\ \underbrace{a_{t+1}}_{\text{Action}},$$

因此得名 **Sarsa**。

从数学上看，Sarsa 是求解给定策略动作价值 Bellman 方程的随机逼近算法。对固定 π，若每个 (s,a) 都满足

$$\sum_t\alpha_t(s,a)=\infty,\qquad\sum_t\alpha_t^2(s,a)<\infty,$$

也就是每个状态–动作对被充分访问，则课本给出的定理说明 $q_t(s,a)$ 几乎必然收敛到 $q_\pi(s,a)$。

7.5.3　怎样用 Sarsa 学策略：评价和改进交替进行

仅有上面的 Sarsa 更新，只能评价一个给定策略。为了搜索更好的策略，课本把它与 ε-greedy 策略改进结合起来：

1. 在 s_t 按当前 ε-greedy 策略选择 a_t；
2. 与环境交互，得到 r_{t+1},s_{t+1}；
3. 在 s_{t+1} 仍按当前策略选择 a_{t+1}；
4. 用五元组 $(s_t,a_t,r_{t+1},s_{t+1},a_{t+1})$ 更新 $q(s_t,a_t)$；
5. 根据新的动作价值，把 s_t 处的策略更新为 ε-greedy，然后继续下一步。

这里没有“把旧策略完全评价准确后才改进”。价值更新一步，策略也随即改进一步，这正是 L5 已出现的 **generalized policy iteration（广义策略迭代）**。使用 ε-greedy 的原因是：同一个策略还要负责生成后续经验，必须保留探索，才能让各个动作有机会被估计。

Sarsa 的目标包含“下一步真的会做什么”

Sarsa 的目标使用 $q_t(s_{t+1}, a_{t+1})$，而 a_{t+1} 是当前行为策略实际选出的动作。若策略会以 ε 的概率探索，这个探索动作也会进入更新目标。因此，Sarsa 评价的正是正在产生数据、也正在被改进的那个策略。

7.6　Expected Sarsa：对下一动作先取策略期望

Sarsa 在 s_{t+1} 只抽取一个实际动作 a_{t+1}。Expected Sarsa 则把下一动作的随机性直接平均掉：

$$\mathbb{E}[q_t(s_{t+1}, A)] = \sum_a \pi_t(a \mid s_{t+1}) q_t(s_{t+1}, a).$$

它的 TD 目标是

$$\boxed{\bar{q}_t^{\text{ExpSarsa}} = r_{t+1} + \gamma \sum_a \pi_t(a \mid s_{t+1}) q_t(s_{t+1}, a)}$$

并仍按

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t \big[\bar{q}_t^{\text{ExpSarsa}} - q_t(s_t, a_t)\big]$$

更新。

与 Sarsa 相比，它需要多做一次对动作的求和，计算量略增；但所涉及的随机变量从

$$\{S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}\} \quad \text{减少为} \quad \{S_t, A_t, R_{t+1}, S_{t+1}\},$$

所以课本指出它可以降低估计方差。

7.7　n-step Sarsa：在一步 TD 与 MC 之间调节

7.7.1　同一个回报可以在不同位置截断

动作价值定义为

$$q_\pi(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a].$$

完整回报可以在第 n 步截断，并把剩余部分换成动作价值：

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n q_\pi(S_{t+n}, A_{t+n}).$$

用实际样本和当前估计替代其中的随机量与真实价值，就得到

$$\boxed{\bar{q}_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n q_t(s_{t+n}, a_{t+n})}.$$

概念上的更新仍然是

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha \big[\bar{q}_t^{(n)} - q(s_t, a_t)\big].$$

7.7.2　为什么更新要延迟 n 步

在时刻 t 还没有观察到 $(r_{t+n}, s_{t+n}, a_{t+n})$，所以必须等到时刻 $t+n$，才能回头更新 (s_t, a_t)。按课本的时间下标，完整写法是

$$q_{t+n}(s_t,a_t) = q_{t+n-1}(s_t,a_t) - \alpha_{t+n-1}(s_t,a_t)\left[q_{t+n-1}(s_t,a_t) - \left(\sum_{k=0}^{n-1}\gamma^k r_{t+k+1} + \gamma^n q_{t+n-1}(s_{t+n},a_{t+n})\right)\right].$$

7.7.3　n 控制了什么

取值	得到的方法	目标与性质
$n=1$	一步 Sarsa	$r_{t+1}+\gamma q(s_{t+1},a_{t+1})$；等待短、方差较低，但更依赖自举初值
一般 n	n-step Sarsa	使用前 n 个实际奖励，再从第 n 步动作价值自举
$n\to\infty$	Monte Carlo	一直使用实际奖励到 episode 结束；课本在统一表达中取 $\alpha=1$，得到完整回报样本

n 较小：更接近 Sarsa，偏差较大、方差较低

n 较大：更接近 MC，偏差较小、方差较高

课本把小 n 时较大的偏差归因于对初始价值猜测的依赖。这里介绍的 n-step Sarsa 本身仍是策略评价方法；要学习更好的策略，仍需像一步 Sarsa 一样加入策略改进。

7.8　Q-learning：直接逼近最优动作价值

7.8.1　把“下一实际动作”换成“下一状态最大动作价值”

Sarsa 求解给定策略的 Bellman 方程。Q-learning 改为求解动作价值形式的 Bellman 最优方程：

$$\boxed{q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q(S_{t+1},a') \,\middle|\, S_t=s, A_t=a\right].}$$

因此它的 TD 目标是

$$\bar{q}_t^{Q} = r_{t+1} + \gamma \max_{a'} q_t(s_{t+1},a'),$$

更新式为

$$\boxed{q_{t+1}(s_t,a_t) = q_t(s_t,a_t) + \alpha_t(s_t,a_t)\left[r_{t+1} + \gamma \max_{a'} q_t(s_{t+1},a') - q_t(s_t,a_t)\right].}$$

同样，只有本次访问的 (s_t,a_t) 被更新。

Q-learning 给定 (s_t,a_t) 后，只需要 (r_{t+1},s_{t+1})；Sarsa 还必须知道行为策略在下一状态实际选择的 a_{t+1}。这一个差别正对应两个算法不同的数学目标。

比较	Sarsa	Q-learning
TD 目标	$r_{t+1}+\gamma q_t(s_{t+1},a_{t+1})$	$r_{t+1}+\gamma \max_{a'} q_t(s_{t+1},a')$
解什么方程	给定策略的 Bellman 方程	Bellman 最优方程
直接估计什么	$q_\pi(s,a)$	最优动作价值
下一动作是否进入样本	需要行为策略实际选出的 a_{t+1}	不需要；更新内部直接取最大值
策略关系	行为策略同时也是被评价、被改进的目标策略	产生动作的行为策略可以不同于由 q 得到的目标策略

7.8.2　为什么仍然需要探索

“行为策略可以是任意策略”不等于“数据覆盖可以任意差”。课本的离策略实验表明：均匀行为策略能频繁访问各状态–动作对，Q-learning 的误差下降较快；当行为策略探索性减弱时，学习速度明显下降。原因是未被充分访问的动作价值没有足够样本可更新。

Q-learning 也会自举，所以初始 $q_0(s,a)$ 会影响收敛速度。课本实验中，初值越接近真实最优价值，收敛越快；但即使初值不准确，只要经验和探索充分，估计仍可逐步改进。

7.9　On-policy、off-policy、online、offline 不要混

7.9.1　先区分两种“策略”

- **行为策略（behavior policy）π_b：**真正与环境交互、生成动作和经验样本的策略；
- **目标策略（target policy）π_T：**正在被评价或不断改进、希望最终学到的策略。

  如果行为策略与目标策略相同，称为 **on-policy**；如果两者不同，称为 **off-policy**。

- **Sarsa 是 on-policy：** a_t 和 a_{t+1} 都由当前策略产生，而 Sarsa 又在评价并改进这个策略；
- **本章的 MC 控制是 on-policy：**被评价和改进的策略就是产生 episode 的策略；
- **Q-learning 具有 off-policy 性质：**它求 Bellman 最优方程，更新目标中的最大化不依赖 π_b 在 s_{t+1} 实际会选什么。因而可以用探索性行为策略收集数据，同时把由当前最优动作价值导出的贪心策略作为目标策略。

Q-learning 的“两条线”

$$\underbrace{\pi_b:\ s_t \longrightarrow a_t \longrightarrow (r_{t+1},s_{t+1})}_{\text{负责产生数据}} \qquad \underbrace{\pi_T(s)=\arg\max_a q(s,a)}_{\text{负责表示要学的策略}}.$$

数据由谁产生和更新最终指向谁，可以是两件事。这正是 off-policy 学习能利用其他策略、甚至预先收集的数据的原因。

on/off-policy 讨论的是**产生数据的策略与学习目标的策略是否相同**；online/offline 讨论的是**数据在学习时是否正在与环境交互产生**：

概念对	判断问题	本章中的含义
on-policy / off-policy	行为策略和目标策略是否相同	Sarsa、MC 是 on-policy；Q-learning 可使用不同的行为与目标策略
online / offline	是否边与环境交互边更新	online 边采样边学；offline 使用预先收集的经验，不在学习时与环境交互

课本同时给出了 Q-learning 的在线实现和使用预先收集 episode 的离线实现。作为 off-policy 方法，Q-learning 允许行为策略和目标策略不同；实际使用时仍必须保证行为策略能提供充分经验。

三个常见误解

- off-policy 不表示“不需要策略”，而是行为策略与目标策略可以不同；
- offline 不表示 off-policy；前者描述数据何时收集，后者描述两种策略的关系；
- Q-learning 的 max 只决定 TD 目标，当前动作 a_t 仍由行为策略产生，仍需探索。

7.10　统一视角：算法的骨架相同，只是 TD 目标不同

对动作价值学习，本章算法都可写成

$$\boxed{q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - \bar{q}_t\right]}$$

或等价地写成

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) + \alpha_t(s_t,a_t)\left[\bar{q}_t - q_t(s_t,a_t)\right].$$

真正区分算法的是 $\bar{q}_t$ 以及它对应的方程：

算法	TD 目标 $\bar{q}_t$	求解对象
Sarsa	$r_{t+1} + \gamma q_t(s_{t+1},a_{t+1})$	给定策略的 Bellman 方程
n-step Sarsa	$\sum_{k=0}^{n-1}\gamma^k r_{t+k+1} + \gamma^n q_t(s_{t+n},a_{t+n})$	给定策略的 n 步 Bellman 形式
Q-learning	$r_{t+1} + \gamma \max_a q_t(s_{t+1},a)$	Bellman 最优方程
Monte Carlo	$r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots$	用完整回报表达的 Bellman 方程

统一地说，这些方法都在用随机逼近求解

$$q(s,a) = \mathbb{E}[\bar{q}_t \mid S_t = s, A_t = a].$$

除 Q-learning 外，表中其他方法求的是给定策略的 Bellman 方程；Q-learning 求的是 Bellman 最优方程。

7.11　拿到一条经验时，怎样判断该用哪个目标

假设手中已有 $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$，只看下一部分如何处理：

- 要评价当前实际执行的策略：用 $q_t(s_{t+1}, a_{t+1})$，得到 Sarsa；
- 要减少下一动作采样带来的随机性：用 $\sum_a \pi_t(a \mid s_{t+1}) q_t(s_{t+1}, a)$，得到 Expected Sarsa；
- 愿意等待 n 步，用更多真实奖励：构造 n-step Sarsa 目标；
- 要直接逼近最优动作价值：用 $\max_a q_t(s_{t+1}, a)$，得到 Q-learning；
- 一直等到 episode 结束、完全不用自举值：使用完整回报，得到 MC。

L7 本章速记

- TD = Bellman 方程 + 经验样本 + 随机逼近。
- 基础 TD：$v(s_t) \leftarrow v(s_t) + \alpha[r_{t+1} + \gamma v(s_{t+1}) - v(s_t)]$。
- TD 可一步后立即更新、能处理 continuing task，并且会 bootstrapping。
- Sarsa 用实际下一动作：$r + \gamma q(s', a')$，评价产生数据的策略。
- Expected Sarsa 对下一动作取策略期望，可降低由 A_{t+1} 带来的方差。
- n-step Sarsa 在一步 Sarsa 与完整回报 MC 之间连接偏差和方差。
- Q-learning 用 $r + \gamma \max_{a'} q(s', a')$，直接求 Bellman 最优方程。
- Sarsa、MC 是 on-policy；Q-learning 允许行为策略与目标策略不同。
- off-policy 不等于不探索；没有充分访问，就没有足够样本更新。
- 所有方法都能读成：“旧估计朝本算法定义的目标移动一小步”。

8　L8：价值函数方法——从表格走向函数逼近

本章要回答的问题

L7 的表格算法要为每个状态或状态–动作对单独保存一个数。当状态很多、连续，或者从未访问过的状态也需要合理估值时，怎样用一个共享参数的函数表示价值？TD、Sarsa、Q-learning 又怎样改写成参数更新，并进一步得到 DQN？

8.1　为什么要把表格换成函数

表格表示为每个状态保存独立条目；函数表示则写成

$$\hat{v}(s,w)\approx v_\pi(s),\qquad \hat{q}(s,a,w)\approx q_\pi(s,a),$$

其中 $w\in\mathbb{R}^m$ 是所有状态共享的参数。最简单的线性形式是

$$\boxed{\hat{v}(s,w)=\phi(s)^\top w},$$

其中 $\phi(s)$ 是状态特征；非线性形式可以是神经网络。

问题	表格表示	函数表示
读取一个值	直接查对应条目	输入状态（和动作）做一次函数计算
更新一个值	只改该条目	改参数 w，可能同时影响许多输入的估值
存储规模	随状态–动作数增长	主要由参数维数决定
未见状态	没有天然泛化	可借共享特征产生近似值
逼近误差	每项可独立表示	受函数族和特征表达能力限制

“函数逼近”既带来泛化，也带来耦合

一次参数更新不再只改变 s_t 的估值。若两个状态的特征相似，更新其中一个也会改变另一个。这正是泛化的来源，也是分析和调参比表格情形更困难的原因。

8.2　把价值估计写成优化问题

给定策略 π，最直观的目标是最小化加权平方价值误差

$$J_E(w)=\mathbb{E}_{S\sim d}\Big[\big(v_\pi(S)-\hat{v}(S,w)\big)^2\Big]=\sum_{s\in\mathcal{S}} d(s)\big(v_\pi(s)-\hat{v}(s,w)\big)^2.$$

这里 d 决定“哪些状态更重要”。课件重点采用策略 π 诱导的平稳分布 d_π：长期更常被访问的状态权重更高。若 P_π 是策略下的状态转移矩阵，则

$$d_\pi^\top P_\pi=d_\pi^\top,\qquad \sum_s d_\pi(s)=1.$$

若真实 v_π 已知，随机梯度更新可以写成

$$w_{t+1} = w_t + \alpha_t\big[v_\pi(s_t) - \hat{v}(s_t, w_t)\big]\nabla_w \hat{v}(s_t, w_t).$$

但强化学习中 v_π 恰好未知，所以需要用可观测目标替代它：完整回报给出 MC 版本，一步自举目标给出 TD 版本。

8.3　带函数逼近的 TD 状态价值学习

定义一步 TD 目标和“目标减当前估计”的残差

$$y_t = r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t), \qquad e_t = y_t - \hat{v}(s_t, w_t).$$

则课件式 (8.13) 的更新为

$$\boxed{w_{t+1} = w_t + \alpha_t e_t \nabla_w \hat{v}(s_t, w_t)}.$$

它仍可读成“让当前输出朝 TD 目标移动”，只是现在真正被更新的是参数，而不是表格中的单个条目。

在线性情形 $\hat{v}(s, w) = \phi(s)^\top w$ 下，$\nabla_w \hat{v}(s, w) = \phi(s)$，因此

$$\boxed{w_{t+1} = w_t + \alpha_t\big[r_{t+1} + \gamma\phi(s_{t+1})^\top w_t - \phi(s_t)^\top w_t\big]\phi(s_t)}.$$

为什么表格 TD 是线性 TD 的特例

令 $\phi(s) = e_s$，即第 s 个分量为 1、其余为 0 的 one-hot 向量。于是 $\phi(s)^\top w = w_s$，而每次更新只会改变 w_s 这一项，正好退化为 L7 的表格 TD。

8.4　线性 TD 最终在求什么

把所有特征按行堆成矩阵 Φ，令 $D = \mathrm{diag}(d_\pi)$，并把 Bellman 方程写成 $v_\pi = r_\pi + \gamma P_\pi v_\pi$。课件对期望更新的整理给出

$$w_{t+1} = w_t + \alpha_t(b - Aw_t),$$

其中

$$\boxed{A = \Phi^\top D(I - \gamma P_\pi)\Phi, \qquad b = \Phi^\top D r_\pi}.$$

在课件使用的平稳采样、线性无关特征等条件下，A 正定且可逆，固定点为

$$\boxed{w^* = A^{-1}b}.$$

若 $\Phi = I$，则 $w^* = v_\pi$；若特征维数较低，通常只能得到价值函数在特征空间中的近似。

为了理解这个近似，定义 Bellman 算子

$$\mathcal{T}_\pi(x) = r_\pi + \gamma P_\pi x$$

和到 range(Φ) 的 D-加权投影

$$M = \Phi(\Phi^\top D\Phi)^{-1}\Phi^\top D.$$

投影 Bellman 误差为

$$J_{\mathrm{PBE}}(w) = \|\Phi w - M\mathcal{T}_\pi(\Phi w)\|_D^2, \qquad \|x\|_D^2 = x^\top D x.$$

课件证明线性 TD 的固定点 $w^* = A^{-1}b$ 正是投影 Bellman 方程的解，也即 J_{PBE} 的极小点。

不要把三个误差混成一个

- J_E：与未知真实价值 v_π 的直接平方误差；
- Bellman error：$\|\Phi w - \mathcal{T}_\pi(\Phi w)\|_D^2$；
- projected Bellman error：先对 Bellman 结果投影回可表示空间再比较。

课件的线性 TD 理论结论针对 projected Bellman error，不能直接说它在普通意义下对 J_E 做精确梯度下降。

8.5　LSTD：直接估计固定点方程

由样本形式

$$A = \mathbb{E}\Big[\phi(s_t)\big(\phi(s_t) - \gamma\phi(s_{t+1})\big)^\top\Big], \qquad b = \mathbb{E}[r_{t+1}\phi(s_t)],$$

可用经验平均得到 $\hat{A}_t, \hat{b}_t$，再令

$$\hat{w}_t = (\hat{A}_t + \sigma I)^{-1}\hat{b}_t, \qquad \sigma > 0.$$

这就是 least-squares TD（LSTD）的核心。它能更直接地利用 $w^* = A^{-1}b$，但要维护矩阵并求逆，计算与存储成本高于普通 TD；课件中的 LSTD 讨论限于线性状态价值估计。

8.6　动作价值：Sarsa 与 Q-learning 的函数版本

把 $q(s,a)$ 换成可微函数 $\hat{q}(s,a,w)$，L7 的两个目标直接变成参数更新。

8.6.1　Sarsa with function approximation

$$\boxed{w_{t+1} = w_t + \alpha_t\big[r_{t+1} + \gamma\hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)\big]\nabla_w\hat{q}(s_t, a_t, w_t).}$$

下一动作 a_{t+1} 由当前策略产生，因此仍是 on-policy；价值更新后再做 ε-greedy 策略改进。

8.6.2　Q-learning with function approximation

$$\boxed{w_{t+1} = w_t + \alpha_t\left[r_{t+1} + \gamma\max_{a'}\hat{q}(s_{t+1}, a', w_t) - \hat{q}(s_t, a_t, w_t)\right]\nabla_w\hat{q}(s_t, a_t, w_t).}$$

它和函数版 Sarsa 的关键差别仍然只有下一状态部分：实际下一动作值换成最大动作值。课件既展示了 on-policy 数据收集版本，也在 DQN 中展示了 off-policy 版本。

8.7　Deep Q-learning：两个网络加经验回放

DQN 用神经网络表示 $\hat{q}(s,a,w)$。对回放样本 (s,a,r,s')，先用参数暂时固定的 target network 构造

$$y_T = r + \gamma \max_{a'} \hat{q}(s', a', w_T),$$

再更新 main network 以减小 mini-batch 损失

$$\boxed{L(w) = \frac{1}{|\mathcal{B}|} \sum_{(s,a,r,s')\in\mathcal{B}} \big[y_T - \hat{q}(s,a,w)\big]^2}.$$

训练过程中 main network 每轮更新，而 target network 每隔 C 次迭代才同步：

$$w_T \leftarrow w.$$

机制	在课件算法中的作用
Main network	输出当前估计并通过 mini-batch 反向传播更新
Target network	在一段时间内固定 TD 目标，之后周期性从 main network 同步
Replay buffer	保存行为策略生成的经验，使样本可重复使用
Uniform mini-batch	从回放池随机抽样，打散轨迹中的时间相关性
Off-policy target	数据可来自行为策略；目标仍对下一动作取最大

训练损失小不等于价值一定准确

课件的 DQN 例子专门比较了充分数据与不足数据：网络可能把回放池中的 TD 损失降得很低，但若经验没有充分覆盖重要状态–动作对，真实价值误差仍可能很大。

L8 本章速记

- 函数逼近用共享参数压缩表格，并在状态之间泛化。
- 基础更新：参数沿 (TD 目标 − 当前输出)$\nabla_w \hat{v}$ 方向移动。
- 线性 TD：$\hat{v} = \phi^\top w$，one-hot 特征退化为表格 TD。
- 平稳线性情形的固定点满足 $Aw^* = b$，对应投影 Bellman 方程。
- Sarsa 与 Q-learning 的函数版只需把表格值替换为 $\hat{q}(s,a,w)$ 并乘输出梯度。
- DQN 的两项关键实现技术是 target network 与 experience replay。
- 小 TD loss 只说明拟合当前数据好，不保证数据覆盖充分或真实价值误差小。

9　L9：策略梯度方法——直接优化参数化策略

本章要回答的问题

L1–L8 都主要通过学习价值再间接改进策略。能否把策略本身写成可微函数 $\pi(a \mid s, \theta)$，定义一个衡量整体表现的标量目标 $J(\theta)$，然后直接沿梯度方向更新策略参数?

9.1　从策略表格到策略函数

函数策略写成

$$\boxed{\pi(a \mid s, \theta)}, \qquad \theta \in \mathbb{R}^m.$$

输入可以是 (s, a) 并输出一个动作概率，也可以输入 s 一次输出所有动作的概率。与 L8 类似，更新不再逐项改表，而是改变共享参数 θ。

参数化后，“最优”先要说明优化哪个标量

表格最优策略可以表述为同时最大化每个状态的价值；参数化策略族可能受表达能力限制，而梯度优化需要一个标量目标。因此必须先说明状态如何加权、是折扣任务还是平均奖励任务。不同目标对应的状态分布与梯度等式并不完全相同。

离散动作中常用 softmax 策略

$$\boxed{\pi(a \mid s, \theta) = \frac{\exp(h(s, a, \theta))}{\sum_{a'} \exp(h(s, a', \theta))}},$$

其中 h 是动作 preference。softmax 保证每个动作概率为正且总和为 1，因此策略本身具有随机探索性。

9.2　两个标量性能指标

9.2.1　平均状态价值

给定状态权重 d，定义

$$\boxed{\bar{v}_\pi = \sum_s d(s) v_\pi(s) = \mathbb{E}_{S \sim d}[v_\pi(S)]}.$$

若 $d = d_0$ 与策略无关，它可以描述固定初始状态分布；若 $d = d_\pi$，则按策略的长期平稳访问频率加权。折扣回报形式

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right]$$

正是某个初始分布 d 下的平均状态价值。

9.2.2　平均一步奖励

令

$$r_\pi(s) = \sum_a \pi(a \mid s, \theta) r(s, a),$$

则长期平均奖励为

$$\boxed{\bar{r}_\pi = \sum_s d_\pi(s) r_\pi(s) = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right]}.$$

在教材讨论的折扣平稳情形中，两类指标满足

$$\bar{r}_\pi = (1-\gamma)\bar{v}_\pi,$$

所以可被同时最大化；但推导梯度时仍必须区分折扣/未折扣和状态分布的选择。

9.3　策略梯度定理

L9 最重要的结论是：多种指标的梯度都可以统一写成

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta,\, A\sim\pi(\cdot|S,\theta)}\left[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)\right]}.$$

展开形式是

$$\nabla_\theta J(\theta) = \sum_s \eta(s) \sum_a \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a).$$

两者等价来自 log-derivative identity：

$$\nabla_\theta \pi(a \mid s,\theta) = \pi(a \mid s,\theta) \nabla_\theta \ln \pi(a \mid s,\theta).$$

为什么这个定理如此关键

直接对 $J(\theta)$ 求导时，状态分布、未来状态以及价值函数本身都随策略变化；策略梯度定理把复杂依赖整理成一个期望。于是只要能从合适分布采样 (S,A)、估计 $q_\pi(S,A)$，就能构造随机梯度。

9.4　统一公式中的状态分布不能省略

教材定理 9.1 是若干具体情形的总结，J 与 η 要配套理解：

- 折扣情形、固定初始分布 d_0：$J = \bar{v}_{\pi,0} = d_0^\top v_\pi$，$\eta = \rho_\pi$ 是从 d_0 出发的折扣总访问权重；
- 折扣情形、平稳分布 d_π：教材给出 $\nabla \bar{r}_\pi = (1-\gamma)\nabla \bar{v}_\pi$ 的近似策略梯度，γ 越接近 1 近似越准确；
- 未折扣平均奖励情形：$J = \bar{r}_\pi$，$\eta = d_\pi$，对应等式严格成立。

固定初始分布时，折扣总访问权重可写成

$$\rho_\pi(s) = \sum_{s_0} d_0(s_0) \sum_{k=0}^{\infty} \gamma^k [P_\pi^k]_{s_0 s}.$$

它不一定是归一化概率；若统一缩放，梯度方向不变，尺度可并入学习率。

不要背成“任何 J 都配 d_π”

策略梯度的 score-function 结构相似，但不同任务下的 J、η 以及“精确等式/ 近似式”不同。复习证明细节时，应回到教材定理 9.2、9.3、9.5 的具体条件。

9.5　REINFORCE：用完整回报估计动作价值

梯度上升先写成

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t).$$

用一条 on-policy episode 中的样本替代期望，并以回报

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$$

估计 $q_\pi(s_t, a_t)$，得到

$$\boxed{\theta \leftarrow \theta + \alpha G_t \nabla_\theta \ln \pi(a_t \mid s_t, \theta)}.$$

这就是 Monte Carlo policy gradient，也称 REINFORCE。

9.5.1　怎样读这个更新

由

$$\nabla_\theta \ln \pi(a_t \mid s_t, \theta) = \frac{\nabla_\theta \pi(a_t \mid s_t, \theta)}{\pi(a_t \mid s_t, \theta)}$$

可见，$G_t > 0$ 时倾向于提高已采样动作的概率，$G_t < 0$ 时倾向于降低它；回报绝对值越大，更新幅度通常越大。

REINFORCE 的三个边界

- 它用完整 episode 回报，因此属于 Monte Carlo 方法，更新目标不自举；
- 动作按当前 π 采样，所以教材中的基础 REINFORCE 是 on-policy；
- 单条轨迹的 G_t 随机性很强，梯度估计往往方差较大，这正是 L10 引入 baseline 与 critic 的动机。

L9 本章速记

- 策略函数 $\pi(a \mid s, \theta)$ 直接输出动作概率，优化对象是参数 θ。
- 参数化策略要先选标量指标 $J(\theta)$，不能省略状态加权方式。
- 策略梯度定理的骨架：$\mathbb{E}[\nabla_\theta \ln \pi(A \mid S, \theta) q_\pi(S, A)]$。
- J 与状态分布 η 随折扣/未折扣和初始/平稳分布而变。
- REINFORCE 用 G_t 估计动作价值，是 on-policy Monte Carlo 策略梯度。
- 回报为正就提高采样动作概率，回报为负就降低；估计方差通常较大。

10　L10：Actor–Critic——让策略更新与价值估计协同工作

本章要回答的问题

L9 的策略梯度需要 q_π，REINFORCE 用完整回报来估计它，方差较大且必须等到 episode 后部信息到达。能否让一个价值学习器在线估值，并把评价结果交给策略学习器？怎样进一步加入 baseline、离策略样本和确定性策略？

10.1　Actor 与 critic 分别做什么

Actor–critic 不是脱离策略梯度的新体系，而是把两类更新组合起来：

- **Actor**：参数化策略 $\pi(a \mid s, \theta)$，负责产生动作并更新策略；
- **Critic**：参数化价值 $\hat{v}(s, w)$ 或 $\hat{q}(s, a, w)$，负责评价当前策略并提供 actor 的更新信号。

主线可以写成

$$\boxed{\text{策略梯度骨架} + \text{TD 价值估计} = \text{Actor–Critic}}.$$

10.2　QAC：最简单的动作价值 Actor–Critic

策略梯度的样本更新为

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \ln \pi(a_t \mid s_t, \theta_t) \hat{q}(s_t, a_t, w_t).$$

Q actor–critic（QAC）用函数版 Sarsa 作为 critic：

$$\begin{aligned} e_t^q &= r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t), \\ w_{t+1} &= w_t + \alpha_w e_t^q \nabla_w \hat{q}(s_t, a_t, w_t). \end{aligned}$$

其中 a_t, a_{t+1} 都由当前策略生成，所以教材算法 10.1 是 on-policy。

方法	$q_\pi(s_t, a_t)$ 的近似	结果
REINFORCE	完整回报 G_t	MC policy gradient
QAC	Sarsa/TD 学到的 $\hat{q}(s_t, a_t, w_t)$	Actor–Critic

10.3　Baseline invariance：改变方差，不改变期望梯度

只要 $b(s)$ 不依赖动作，就有

$$\boxed{\mathbb{E}[\nabla_\theta \ln \pi(A \mid S, \theta) q_\pi(S, A)] = \mathbb{E}\big[\nabla_\theta \ln \pi(A \mid S, \theta)\big(q_\pi(S, A) - b(S)\big)\big]}.$$

关键等式是

$$\begin{aligned} \mathbb{E}_{A \sim \pi(\cdot \mid s)}[\nabla_\theta \ln \pi(A \mid s, \theta) b(s)] &= b(s) \sum_a \nabla_\theta \pi(a \mid s, \theta) \\ &= b(s) \nabla_\theta 1 = 0. \end{aligned}$$

因此 baseline 不改变期望梯度，却会改变单样本梯度的方差。

教材给出的最小方差 baseline 为

$$b^*(s) = \frac{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta \ln \pi(A \mid s,\theta)\|^2 q_\pi(s,A)]}{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta \ln \pi(A \mid s,\theta)\|^2]},$$

但它难以直接计算。去掉梯度范数权重可得更实用的基线

$$b(s) = \mathbb{E}_{A\sim\pi}[q_\pi(s,A)] = v_\pi(s).$$

10.4　A2C：用优势函数更新 actor

优势函数衡量某动作相对该状态平均水平的好坏：

$$\boxed{A_\pi(s,a) = q_\pi(s,a) - v_\pi(s)}.$$

若 $A_\pi(s,a) > 0$，该动作优于策略在此状态下的平均动作；若小于 0，则较差。使用状态价值作 baseline 后，actor 更新为

$$\theta_{t+1} = \theta_t + \alpha_\theta \hat{A}_t \nabla_\theta \ln \pi(a_t \mid s_t,\theta_t).$$

教材算法 10.2 用一步 TD 信号近似优势：

$$\boxed{\delta_t^+ = r_{t+1} + \gamma \hat{v}(s_{t+1},w_t) - \hat{v}(s_t,w_t)}.$$

于是 A2C（也称 TD actor–critic）的两次更新是

$$\boxed{\begin{aligned}\theta_{t+1} &= \theta_t + \alpha_\theta \delta_t^+ \nabla_\theta \ln \pi(a_t \mid s_t,\theta_t),\\ w_{t+1} &= w_t + \alpha_w \delta_t^+ \nabla_w \hat{v}(s_t,w_t).\end{aligned}}$$

教材内部有两种 TD error 符号约定

L7 把 TD error 定义成“当前估计减目标”，即 $\delta_t^- = \hat{v}(s_t) - [r_{t+1} + \gamma \hat{v}(s_{t+1})]$，并用 $-\alpha\delta_t^-$ 更新；L10 的 A2C 课件把它定义成“目标减当前估计”，即上式的 $\delta_t^+ = -\delta_t^-$，并用 $+\alpha\delta_t^+$ 更新。数值更新完全相同。复习时先看定义，再看更新式前的正负号。

10.5　Off-policy Actor–Critic：用重要性采样校正动作分布

设目标策略为 π，产生数据的行为策略为 β。重要性采样的基本恒等式是

$$\mathbb{E}_{X\sim p_0}[f(X)] = \mathbb{E}_{X\sim p_1}\left[\frac{p_0(X)}{p_1(X)} f(X)\right].$$

对动作分布，对应权重为

$$\boxed{\rho_t = \frac{\pi(a_t \mid s_t,\theta_t)}{\beta(a_t \mid s_t)}}.$$

必须满足覆盖条件：只要 $\pi(a \mid s) > 0$，就要有 $\beta(a \mid s) > 0$，否则比值无定义且目标动作无法从数据中恢复。

教材定理 10.1 以

$$J(\theta) = \sum_s d_\beta(s) v_\pi(s)$$

为目标，并将离策略梯度写成带 ρ_t 的期望。使用状态价值 baseline 与一步 TD 优势后，教材算法 10.3 为

$$\boxed{\begin{aligned} \theta_{t+1} &= \theta_t + \alpha_\theta \rho_t \delta_t^+ \nabla_\theta \ln \pi(a_t \mid s_t, \theta_t), \\ w_{t+1} &= w_t + \alpha_w \rho_t \delta_t^+ \nabla_w \hat{v}(s_t, w_t). \end{aligned}}$$

这里 actor 与 critic 都使用重要性权重；这是本教材给出的算法形式。

重要性采样的代价

它校正分布偏差，但当 π/β 很大时会放大方差。覆盖不足不是靠权重就能补救的；行为策略仍需对目标策略可能选择的动作提供样本。

10.6　Deterministic Actor–Critic

随机策略输出动作分布；确定性策略直接输出动作：

$$\boxed{a = \mu(s, \theta)}.$$

这对连续动作空间尤其自然。教材的 deterministic policy gradient theorem 给出

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta}\Big[\nabla_\theta \mu(S,\theta)\ \nabla_a q_\mu(S,a)|_{a=\mu(S,\theta)}\Big]}.$$

链式法则可这样读：critic 先告诉 actor “动作往哪个方向改变能提高 q”；actor 再把动作梯度通过 $\mu(s,\theta)$ 传回参数。

用行为策略 β 产生 a_t，而用目标策略 μ 构造下一动作，教材算法 10.4 的 TD 信号和更新为

$$\begin{aligned} e_t^q &= r_{t+1} + \gamma \hat{q}(s_{t+1}, \mu(s_{t+1}, \theta_t), w_t) - \hat{q}(s_t, a_t, w_t), \\ \theta_{t+1} &= \theta_t + \alpha_\theta \nabla_\theta \mu(s_t, \theta_t)\ \nabla_a \hat{q}(s_t, a, w_t)|_{a=\mu(s_t,\theta_t)}, \\ w_{t+1} &= w_t + \alpha_w e_t^q \nabla_w \hat{q}(s_t, a_t, w_t). \end{aligned}$$

动作 a_t 用于实际交互，$\mu(s_{t+1})$ 只用于计算 TD 目标。行为策略可以是任意充分探索的策略，也可以通过给 μ 的动作加噪声得到。

算法	Actor 信号	Critic	数据策略
QAC	$\hat{q}(s,a)$	Sarsa 动作价值	on-policy
A2C	$\delta_t^+ \approx A_\pi(s,a)$	TD 状态价值	on-policy
Off-policy AC	$\rho_t \delta_t^+$	重要性加权 TD	$\beta \neq \pi$
Deterministic AC	$\nabla_a \hat{q}$ 经 μ 链式回传	动作价值 TD	通常 off-policy

L10 本章速记

- Actor 更新策略，critic 用 MC/TD 估值；采用 TD critic 时通常称 actor–critic。
- QAC 直接把 $\hat{q}(s,a)$ 放进策略梯度，critic 使用 Sarsa。
- 与动作无关的 baseline 不改变期望梯度，但能改变方差。
- $v_\pi(s)$ 是实用 baseline，得到优势 $A_\pi = q_\pi - v_\pi$。
- A2C 用一步 TD 信号同时更新 actor 和状态价值 critic。
- 离策略 AC 用 π/β 校正动作采样分布，并要求行为策略覆盖目标策略。
- 确定性策略梯度沿 critic 的动作梯度改进连续动作。

11 L1–L10 一页串联

十讲之间的依赖关系

章节	核心问题	和后面内容的关系
L1	如何把交互问题写成 MDP	给出状态、动作、奖励、模型、策略、回报这些基本语言
L2	固定一个策略，它到底好不好	用 Bellman 期望方程定义并计算 v_π, q_π
L3	什么叫最优策略与最优价值	把“按策略平均”变成“对动作取最大”
L4	模型已知时怎样求最优策略	VI/PI 用模型计算 Bellman 更新
L5	模型未知时怎样继续做控制	MC 用 episode 回报的样本均值估计 $q(s,a)$
L6	样本式迭代为什么可能收敛	随机逼近与 SGD 解释“旧估计朝目标移动”的更新模板
L7	不等 episode 结束，怎样用一步或多步经验学习	TD、Sarsa 与 Q-learning 把随机逼近用于 Bellman 方程
L8	状态太多时怎样表示和学习价值	用参数化函数共享信息，并由函数版 TD/Q-learning 走向 DQN
L9	怎样不经过贪心值表而直接改进策略	参数化策略、策略梯度定理与 REINFORCE 建立 policy-based 主线
L10	怎样让策略学习与价值学习互相配合	Actor–Critic 用 critic 提供的价值、优势或动作梯度更新 actor

只保留十组公式

1. **回报递推**

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

2. **固定策略的价值**

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_\pi(s')].$$

3. **最优价值**

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_*(s')].$$

4. **算法分工**

$$\text{VI:} \quad v_{k+1} = \mathcal{T}_* v_k,$$

$$\text{PI:} \quad \pi_k \xrightarrow{\text{评价}} v_{\pi_k} \xrightarrow{\text{贪心改进}} \pi_{k+1}.$$

5. **Monte Carlo 估计动作价值**

$$\hat{q}_\pi(s,a) = \frac{1}{N} \sum_{i=1}^{N} G^{(i)}(s,a).$$

6. **随机逼近/SGD 的共同更新形状**

$$w_{k+1} = w_k + \alpha_k(\text{目标信号} - w_k), \qquad \sum_k \alpha_k = \infty, \quad \sum_k \alpha_k^2 < \infty.$$

7. **TD 动作价值算法的统一骨架**

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t\big[\bar{q}_t - q_t(s_t, a_t)\big],$$

$$\bar{q}_t = \begin{cases} r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}), & \text{Sarsa,} \\ \displaystyle\sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n q_t(s_{t+n}, a_{t+n}), & n\text{-step Sarsa,} \\ r_{t+1} + \gamma \max_a q_t(s_{t+1}, a), & \text{Q-learning.} \end{cases}$$

8. **函数逼近 TD**

$$w_{t+1} = w_t + \alpha_t\big[r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\big]\nabla_w \hat{v}(s_t, w_t).$$

9. **策略梯度与 REINFORCE**

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \ln \pi(A \mid S, \theta) q_\pi(S, A)], \qquad \theta \leftarrow \theta + \alpha G_t \nabla_\theta \ln \pi(a_t \mid s_t, \theta).$$

10. **A2C**

$$\delta_t^+ = r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t),$$

$$\theta_{t+1} = \theta_t + \alpha_\theta \delta_t^+ \nabla_\theta \ln \pi(a_t \mid s_t, \theta_t), \qquad w_{t+1} = w_t + \alpha_w \delta_t^+ \nabla_w \hat{v}(s_t, w_t).$$

十九个自测问题

1. 为什么奖励不等于价值?
2. 为什么 $v_\pi(s)$ 是各个 $q_\pi(s, a)$ 的加权平均?
3. Bellman 期望方程中的“平均”为什么在最优方程中变成了“最大”?
4. 为什么 $\gamma < 1$ 不只是在表达“重视眼前”?
5. 价值迭代与策略迭代最本质的区别是什么?
6. 为什么无模型 MC 控制通常直接估计 $q(s, a)$?
7. exploring starts 和 ε-greedy 都在解决什么问题?
8. 随机逼近的步长为什么既要满足 $\sum \alpha_k = \infty$，又要满足 $\sum \alpha_k^2 < \infty$?
9. TD 为什么不必像 MC 那样等 episode 结束?
10. Sarsa 的名字来自哪五个量，它的 TD 目标是什么?
11. n-step Sarsa 中的 n 怎样连接 Sarsa 与 MC?
12. Sarsa 与 Q-learning 的更新只差哪里? 它们分别求什么方程?

13. on-policy/off-policy 与 online/offline 分别在区分什么?
14. 为什么函数逼近既能泛化，又会让一次更新影响多个状态?
15. 线性 TD 的固定点 $w^* = A^{-1}b$ 对应哪一种 Bellman 误差?
16. DQN 为什么同时需要 main network、target network 与 replay buffer?
17. 策略梯度定理中的 J 与状态分布 η 为什么必须配套说明?
18. 为什么减去只依赖状态的 baseline 不改变期望策略梯度?
19. A2C、离策略 AC 和确定性 AC 分别用什么信号更新 actor?

自测答案关键词

1. 奖励只属于一步；价值是未来折扣奖励的期望总和。
2. 当前动作由策略按 $\pi(a \mid s)$ 随机选择，因此使用全期望公式。
3. 最优控制允许在每个状态选择动作价值最大的动作。
4. 它还保证有界回报、Bellman 算子的压缩性和迭代收敛。
5. VI 每轮只做一步最优备份；PI 先充分评价一个策略，再贪心改进。
6. 模型未知时，无法由 $v(s')$ 可靠展望每个动作；$q(s,a)$ 能直接比较动作。
7. 都在保证每个状态–动作对有机会被尝试，避免漏掉潜在好动作。
8. 前者保证总路程足够，后者让噪声的长期影响可控。
9. TD 用一步奖励加下一状态的当前价值估计构造目标，可立即自举更新。
10. State–Action–Reward–State–Action；目标为 $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$。
11. $n = 1$ 是一步 Sarsa；$n \to \infty$ 时使用完整回报，成为 MC。
12. 前者用实际下一动作的价值并求给定策略的 Bellman 方程；后者用下一状态动作价值的最大值并求 Bellman 最优方程。
13. 前者比较行为策略和目标策略是否相同；后者比较学习时是否正在与环境交互并产生数据。
14. 参数由所有输入共享；相似状态使用重叠特征，因而既共享经验也共享参数改变量。
15. 对应 projected Bellman error，而不是直接对未知真实价值平方误差做精确梯度下降。
16. main network 拟合当前值；target network 暂时固定目标；replay buffer 打散轨迹相关性并重复利用经验。
17. 不同折扣/平均奖励目标诱导不同的访问权重，统一梯度公式中的 η 与等式是否精确也随之改变。
18. 因为对每个状态都有 $\mathbb{E}_{A\sim\pi}[\nabla_\theta \ln \pi(A \mid s) b(s)] = b(s)\nabla_\theta \sum_a \pi(a \mid s) = 0$。

19. A2C 用 TD 优势 δ_t^+；离策略 AC 用重要性加权的 $\rho_t\delta_t^+$；确定性 AC 用 critic 的动作梯度 $\nabla_a\hat{q}$ 经策略函数反向传播。

内容核对说明

本文以本目录中的 L1–L10 十份课件为章节依据，并与仓库根目录中对应的十章教材 PDF 交叉核对。L7–L10 的直接核对材料为 `L7-Temporal-Difference Learning.pdf`、`L8-Value function methods.pdf.pdf`、`L9-Policy gradient methods.pdf`、`L10-Actor Critic.pdf`，以及仓库根目录中教材第 7–10 章的对应 PDF。新增内容分别覆盖教材第 8.1–8.5、9.1–9.5、10.1–10.5 节及对应 slide；总结中的公式、术语与算法边界均按这些材料统一。核对时特别处理了以下容易造成误解的边界条件：

- 用联合模型 $p(s', r \mid s, a)$ 表示最一般的单步随机性；
- 明确 Bellman 算子的压缩性使用无穷范数；
- 区分“最优价值唯一”与“最优策略不一定唯一”；
- 为奖励平移不变性补充无限时域折扣任务这一适用语境；
- 明确 VI/PI 属于模型已知时的动态规划方法；
- 明确 MC 方法不需要模型，但需要可采样的 episode 和足够探索；
- 明确固定 $\varepsilon > 0$ 的 ε-greedy 一般优化的是软策略集合中的表现，不等同于无约束最优确定性策略；
- 明确随机逼近/SGD 的收敛依赖步长、噪声和样本假设，不能把样本更新理解为无条件收敛。
- 保留教材各章原有的 TD error 约定：L7 使用“当前估计减目标”并配负号更新，L10 使用“目标减当前估计”并配正号更新，同时明确两式数值等价；
- 明确基础 TD 和固定策略 Sarsa 的收敛定理要求逐状态或逐状态–动作对满足步长条件及充分访问；
- 区分 Sarsa 的实际下一动作目标、Expected Sarsa 的策略期望目标、Q-learning 的最大化目标；
- 区分 on-policy/off-policy 与 online/offline，并强调 off-policy Q-learning 仍需要具有充分探索性的经验。
- 明确函数逼近更新改变共享参数，表格 TD 是 one-hot 线性 TD 的特例；
- 区分真实价值误差、Bellman error 与 projected Bellman error，并把 $w^* = A^{-1}b$ 限定在线性、平稳采样与相应秩条件下；
- 按课件区分 DQN 的 main/target network 与 replay buffer，并说明训练 TD 损失小不代表经验覆盖充分；
- 对策略梯度统一式补充 J、η、折扣/未折扣及精确/近似条件，避免把所有情形都简写成同一个平稳分布；

- 明确 baseline 必须不依赖动作才能保持期望策略梯度不变；
- 对离策略 actor–critic 补充重要性比率的支持集条件与方差代价；
- 区分随机策略梯度中的 score function 与确定性策略梯度中的动作梯度。

最容易混淆的概念

概念对	前者	后者
奖励 / 回报	一步交互后的即时反馈	从现在起整条未来轨迹的折扣奖励和
回报 / 价值	一次具体轨迹上的随机结果	给定起点后，对所有可能回报取期望
v_π / q_π	从状态开始，第一步也服从 π	从状态–动作对开始，第一步动作已指定
Bellman 期望 / 最优方程	固定策略，对动作按 π 求平均	寻找最优策略，对动作取最大
策略评价 / 策略改进	策略不变，计算它的价值	利用动作价值改变策略
v_k / v_{π_k}	算法第 k 轮的价值近似	策略 π_k 的精确价值
有模型 / 无模型	知道转移概率和奖励分布，可直接计算期望	不知道模型，只能从交互样本估计价值
Exploring starts / ε-greedy	要求 episode 能从任意 (s,a) 开始	通过软策略让每个动作持续有正概率被选到
MC 估计 / 随机逼近	用回报样本平均估计价值	研究带噪声观测下的迭代更新和收敛条件
MC / TD	等 episode 结束，用完整回报且不自举	一步或多步后即可更新，目标中使用当前价值估计
Sarsa / Q-learning	用实际下一动作，求给定策略的 Bellman 方程	对下一动作取最大，求 Bellman 最优方程
on-policy / off-policy	行为策略与目标策略相同	行为策略与目标策略不同
online / offline	边与环境交互边更新	用预先收集的数据更新
BGD / SGD	每轮使用全部样本，方向稳定但计算贵	每轮使用单个样本，便宜但随机性更强

最容易混淆的概念（续）

概念对	前者	后者
表格 / 函数逼近	每个状态–动作条目独立存储	共享参数、可泛化，但一次更新会耦合多个输入
真实价值误差 / 投影 Bellman 误差	直接与未知 v_π 比较	把 Bellman 结果投影回可表示空间后比较
Main / target network	每轮拟合当前动作价值	暂时固定 TD 目标并周期性同步
Value-based / policy-based	先估价值再贪心或软改进策略	直接对参数化策略的标量指标求梯度
REINFORCE / A2C	完整回报估计 q_π，Monte Carlo	TD error 近似优势，actor 与 critic 在线协同
QAC / A2C	critic 学 $q_\pi(s,a)$，actor 直接用动作价值	critic 学 $v_\pi(s)$，actor 使用优势/TD error
随机 / 确定性策略梯度	用 $\nabla_\theta \ln \pi(a \mid s, \theta)$	用 $\nabla_\theta \mu(s) \nabla_a q_\mu(s,a)$

建议复习顺序：先看每章橙色“本章速记”，再看“一页串联”；
价值类公式先回到“当前奖励 $+\gamma\times$ 后续价值”；
策略类公式先回到“梯度方向 = 策略敏感度 $\times$ 价值/优势”。